hw4

October 13, 2026

[ ]:
```python
# Question 1

import numpy as np


def get_velocity(mu, r, a):
    return np.sqrt(mu * (2/r - 1/a))

def get_period(mu, a):
    return 2 * np.pi * np.sqrt(a**3/mu)


def time_to_get_point_dist_point_circ(dist, velocity, radius, Earth_Mu = None,␣
 ↪n_rev = 2):
    circum = 2 * np.pi * radius
    arc_dist = (dist/circum) * 2*np.pi
    period_circ = get_period(Earth_Mu, radius)
    total_arc_length = (n_rev * np.pi * 2)  + arc_dist  # need to go slower
    time_arc = period_circ * (total_arc_length/(2*np.pi))

    return time_arc

def get_P_phasing(time, revolutions):
    return (time/revolutions) # we divide by revolutions because we want to␣
 ↪gain total_time/rev per rev

def get_a_from_P(mu, P):
    return ((P/(2*np.pi))**2 * mu)**(1/3)
```

[ ]:
```python
Earth_Mu = 398600.0 # km^3/s^2
R_earth = 6378.0 # km
r_7 = 272.0 + R_earth

#to find velocity we apply vis-viva
a = r_7 # circular orbit
v = get_velocity(Earth_Mu, r_7, a)
P = get_period(Earth_Mu, a)
```

```
time = time_to_get_point_dist_point_circ(500, v, r_7, Earth_Mu, 2)
P_phasing_7 = get_P_phasing(time, 2)
a_phasing_7 = get_a_from_P(Earth_Mu, P_phasing_7)
v_phasing_7 = get_velocity(Earth_Mu, r_7, a_phasing_7)
delta_v_7 = np.abs(v_phasing_7 * 1000 - v * 1000) * 2 # m/s

vals = [a, v, P/60, time/60, P_phasing_7, a_phasing_7, v_phasing_7, delta_v_7]
vals_names = ["a", "v", "P", "time", "P_phasing_7", "a_phasing_7",␣
 ↪"v_phasing_7", "delta_v_7"]

for i  in range(len(vals)):
    if type(vals[i]) ==  np.float64 or type(vals[i]) == float:
        print(f"{(vals_names[i])} = {vals[i]:.3f}")
        print(f"\n")
    else:
        var = vals[i]
        print(f"{(vals_names[i])} = [{var[0]:.3f}, {var[1]:.3f}, {var[2]:.
 ↪3f},]")
        print(f"\n")
```

[ ]:
```
Earth_Mu = 398600.0 # km^3/s^2
R_earth = 6378.0 # km
r_6 = 272.0 + R_earth

#to find velocity we apply vis-viva
a = r_6 # circular orbit
v = get_velocity(Earth_Mu, r_6, a)
P = get_period(Earth_Mu, a)
time = time_to_get_point_dist_point_circ(-500, v, r_6, Earth_Mu)
P_phasing_6 = get_P_phasing(time, 2)
a_phasing_6 = get_a_from_P(Earth_Mu, P_phasing_6)
v_phasing_6 = get_velocity(Earth_Mu, r_6, a_phasing_6)
delta_v_6 = np.abs(v_phasing_6 * 1000 - v * 1000) * 2 # m/s multiply 2 since␣
 ↪two burns

vals = [a, v, P/60, time/60, P_phasing_6, a_phasing_6, v_phasing_6, delta_v_6]
vals_names = ["a", "v", "P", "time", "P_phasing_6", "a_phasing_6",␣
 ↪"v_phasing_6", "delta_v_6"]

for i  in range(len(vals)):
    if type(vals[i]) ==  np.float64 or type(vals[i]) == float:
        print(f"{(vals_names[i])} = {vals[i]:.3f}")
        print(f"\n")
    else:
        var = vals[i]
        print(f"{(vals_names[i])} = [{var[0]:.3f}, {var[1]:.3f}, {var[2]:.
 ↪3f},]")
```

```
        print(f"\n")
```

[ ]:
```
### Question 2

def get_major_axis_from_rp_ra(rp, ra):
    return (rp + ra)/2

def get_period_from_rp_ra(mu, rp, ra):
    a = get_major_axis_from_rp_ra(rp, ra)
    return np.pi * 2 * np.sqrt(a**3/mu)

def get_velocity(mu, r, a):
    return np.sqrt(mu * (2/r - 1/a))

def get_inclination_delta_v(inc1, inc2, v):
    return 2 * v * np.sin(np.radians(np.abs(inc2 - inc1)/2))

def get_arbitrary_delta_v(v_i, v_f, inc1, inc2):
    delta_v = v_i ** 2
    delta_v += v_f **2
    delta_v -= 2 * v_i * v_f * np.cos(np.radians(np.abs(inc2 - inc1)))
    return np.sqrt(delta_v)
```

[ ]:
```
Earth_Mu = 398600.0 # km^3/s^2
R_earth = 6378.0 # km
r_p = 200.0 + R_earth # km
r_a = 42164.0 # km

period = get_period_from_rp_ra(Earth_Mu, r_p, r_a)
time_to_apogee = period/2 # because at true anomaly at 180 degrees we are at␣
 ↪apogee and that is half the period
a = get_major_axis_from_rp_ra(r_p, r_a)
v_a = get_velocity(Earth_Mu, r_a, a)
v_GEO = get_velocity(Earth_Mu, r_a, r_a) # the semi major axis for GEO is r_a =␣
 ↪a since in circular GEO
delta_v = np.abs(v_GEO  - v_a ) # km/s
delta_v_inc = get_inclination_delta_v(28.5, 0, v_GEO)# km/s
total_delta_v = delta_v + delta_v_inc # km/s
arbitrary_delta_v = get_arbitrary_delta_v(v_a, v_GEO, 28.5, 0)

vals = [period, time_to_apogee, a, v_a, v_GEO, delta_v, delta_v_inc,␣
 ↪total_delta_v, arbitrary_delta_v]
vals_names = ["period", "time_to_apogee", "a", "v_a", "v_GEO", "delta_v",␣
 ↪"delta_v_inc", "total_delta_v", "arbitrary_delta_v"]

for i  in range(len(vals)):
    if type(vals[i]) ==  np.float64 or type(vals[i]) == float:
```

```
        print(f"{(vals_names[i])} = {vals[i]:.3f}")
        print(f"\n")
    else:
        var = vals[i]
        print(f"{(vals_names[i])} = [{var[0]:.3f}, {var[1]:.3f}, {var[2]:.
↪3f},]")
        print(f"\n")
```

[ ]:
```
### Question 3

def get_semi_major_axis(ra, rp):
    return (ra + rp)/2

def get_period(mu, a):
    return 2 * np.pi * np.sqrt(a**3/mu)

def get_velocity(mu, r, a):
    return np.sqrt(mu * (2/r - 1/a))
```

[ ]:
```
Earth_Mu = 398600.0 # km^3/s^2
r_p = 7000 # km
r_a = 384400 # km

a_transfer = get_semi_major_axis(r_a, r_p)
transfer_period = get_period(Earth_Mu, a_transfer)/2 # division by two since␣
↪only going half way
velocity_perigee = get_velocity(Earth_Mu, r_p, r_p) # circular orbit
velocity_transfer_perigee = get_velocity(Earth_Mu, r_p, a_transfer)
velocity_transfer_apogee = get_velocity(Earth_Mu, r_a, a_transfer)
velocity_apogee = get_velocity(Earth_Mu, r_a, r_a) # circular orbit

delta_v_1 = np.abs(velocity_transfer_perigee - velocity_perigee)
delta_v_2 = np.abs(velocity_apogee - velocity_transfer_apogee)
total_delta_v = delta_v_1 + delta_v_2

vals = [a_transfer, transfer_period/ (3600 * 24), velocity_perigee,␣
↪velocity_transfer_perigee, velocity_transfer_apogee, velocity_apogee,␣
↪delta_v_1, delta_v_2, total_delta_v]
vals_names = ['a_transfer', 'transfer_period', 'velocity_perigee',␣
↪'velocity_transfer_perigee', 'velocity_transfer_apogee', 'velocity_apogee',␣
↪'delta_v_1', 'delta_v_2', 'total_delta_v']


for i  in range(len(vals)):
    if type(vals[i]) ==  np.float64 or type(vals[i]) == float:
        print(f"{(vals_names[i])} = {vals[i]:.3f}")
        print(f"\n")
```

```
    else:
        var = vals[i]
        print(f"{(vals_names[i])} = [{var[0]:.3f}, {var[1]:.3f}, {var[2]:.
↪3f},]")
        print(f"\n")
```

[ ]:
```
Earth_Mu = 398600.0 # km^3/s^2
r_A = 7000 # km
r_C = 384400 # km
r_B = 600000 # km

a_transfer_1 = get_semi_major_axis(r_A, r_B)
velocity_perigee = get_velocity(Earth_Mu, r_A, r_A) # circular orbit
print(velocity_perigee)
velocity_transfer_perigee_1 = get_velocity(Earth_Mu, r_A, a_transfer_1)
time_1 = get_period(Earth_Mu, a_transfer_1)/2 # half way

a_transfer_2 = get_semi_major_axis(r_B, r_C)
velocity_transfer_apogee_1 = get_velocity(Earth_Mu, r_B, a_transfer_1)
velocity_transfer_apogee_2 = get_velocity(Earth_Mu, r_B, a_transfer_2)
time_2 = get_period(Earth_Mu, a_transfer_2)/2 # half way

velocity_transfer_perigee_2 = get_velocity(Earth_Mu, r_C, a_transfer_2)
velocity_final = get_velocity(Earth_Mu, r_C, r_C)


delta_v_1 = np.abs(velocity_transfer_perigee_1 - velocity_perigee)
delta_v_2 = np.abs(velocity_transfer_apogee_2 - velocity_transfer_apogee_1)
delta_v_3 = np.abs(velocity_final - velocity_transfer_perigee_2)

print(f"delta_v_1: {delta_v_1}")
print(f"delta_v_2: {delta_v_2}")
print(f"delta_v_3: {delta_v_3}")

total_delta_v = delta_v_1 + delta_v_2 + delta_v_3
total_time = time_1 + time_2

vals = [a_transfer_1, velocity_perigee, velocity_transfer_perigee_1,␣
↪a_transfer_2, velocity_transfer_apogee_1,  velocity_transfer_apogee_2,␣
↪velocity_transfer_perigee_2, velocity_final, total_delta_v, total_time/(3600␣
↪* 24)]
vals_names = ['a_transfer_1', 'velocity_perigee',␣
↪'velocity_transfer_perigee_1', 'a_transfer_2', 'velocity_transfer_apogee_1',␣
↪ 'velocity_transfer_apogee_2', 'velocity_transfer_perigee_2',␣
↪'velocity_final', 'total_delta_v', 'total_time']
```

```
for i  in range(len(vals)):
    if type(vals[i]) ==  np.float64 or type(vals[i]) == float:
        print(f"{(vals_names[i])} = {vals[i]:.3f}")
        print(f"\n")
    else:
        var = vals[i]
        print(f"{(vals_names[i])} = [{var[0]:.3f}, {var[1]:.3f}, {var[2]:.
↪3f},]")
        print(f"\n")
```